

A New Modified Sorting Algorithm:

A Comparison with State of the Art

Adam Kearns⁽²³³²⁶⁹⁰³⁾, Cormac Kehoe⁽²³⁴¹¹³⁰²⁾, William Lin⁽²³³¹⁴²¹³⁾, Liam Maher⁽²³⁴⁴⁴⁸⁸²⁾
University College Dublin, Dublin, Ireland



I. Introduction:

The algorithm we implemented (based off the paper *A New Modified Sorting Algorithm: A Comparison with State of the Art*) presents a new sorting algorithm for sorting all integers ($x \in \mathbb{Z}$) both positive and negative. The group essentially took the structure of an established sorting method (*CountingSort*), expanded it to support a larger set of inputs ($\pm x \in \mathbb{Z}$), optimized it for time, and subsequently validated it with testing. The proposed algorithm consistently outperformed most others and notably outperformed QuickSort for a large amount of inputs (100,000 input values), running at a time complexity of $O(n + k)$ or in many cases as low as $O(n)$. Overall the new algorithm proved to appear simple, fast, and scalable.

II. Proposed Method Name: (Up to 1 Page)

The paper is organized into six clear sections, each serving its own purpose effectively. The proposed algorithm is well motivated by real world needs for speed. However, certain methodological details could be clearer, and the lack of stability and testing ranges limits practical insight.

The proposed algorithm in the paper is structured similarly to *CountingSort* but differs fundamentally. It has four arrays, handles negatives using index manipulation techniques, and crucially for its ultimate run time, no comparisons are made between elements. The algorithms performance is tested against other various algorithms for an array of values $-8 \leq x \leq 8$.

The paper benchmarked their algorithm in C# on a Pentium E5300^(2.6 GHz, 4 GB RAM), comparing it against QuickSort, BubbleSort, HeapSort, InsertionSort, BucketSort, SelectionSort and MergeSort. They measured execution times from 10 up to 100,000 random integer inputs^(in the range -8 - 8) using DateTime.Now and TimeSpan to

report hours, minutes, seconds and milliseconds for each run.

Across all scales, the proposed algorithm remains competitive:

at 10 elements it sorts in 0.002 s versus QuickSort's 0.003 s, and

at 100 elements 0.015 s versus 0.016 s.

At 1 000 elements it records 0.125 s against QuickSort's 0.120 s, and

at 10 000 elements it matches QuickSort (0.97 s each).

Most importantly,

at 100 000 elements it completes in 8.93 s, outperforming QuickSort (10.32 s) and dramatically beating SelectionSort (42.93 s). This confirms its linear-time advantage when the input range is bounded

The algorithm's strong performance on values in the range $-8 \dots 8$ may not generalize to broader or skewed data distributions, where the index-based counting approach could behave differently. Moreover, the paper reports single-run timings without repeated trials or measures of variance, leaving open questions about consistency under fluctuating system conditions. Future evaluations should include multiple runs with statistical metrics (mean, standard deviation) to validate the algorithm's reliability across diverse datasets.

III. Implementation Details: (Up to 2 Pages)

We structured our sorting routines using a single *SortingAlgorithms* interface, which defined `int[] sort(int[])`. This uniform API allowed us to insert each algorithm we used (see *figure 1*) to be plugged into our benchmark test seamlessly and equally the same. The *ProposedAlgorithm* itself follows the paper's two pass design in $O(n + k)$ time.

First, the algorithm performs an initial scan of the input array to identify the minimum and maximum values. Using these, it computes an offset equal to $-min$, which allows all integers ($\pm x \in \mathbb{Z}$) to be mapped into valid array indices. Three counting structures are then initialized: a single counter (*zeroCount*) for the number of zeros, an array

(*countp*) to record the frequency of each positive integer, and another array (*countn*) to record the frequency of each negative integer after applying the offset.

During the second case passing over the inputs, each element increments its corresponding counter depending on whether it is negative, zero, or positive.

Finally, the sorted output array is reconstructed with all negative values being emitted first in increasing order, followed by all zero values, and then ultimately all the positive values, still in ascending order. This design enables the algorithm to achieve sorting in linear time without relying on any element-to-element comparisons.

Our implementation closely follows the original algorithm described in the paper, preserving its two-pass structure and counting-based sorting mechanism. While the paper outlined the method at a high level, we refined it for clarity and efficiency. We explicitly separated the negatives/zeros/positives from one another using distinct counters, ensuring a clean reconstruction phase.

While the ProposedAlgorithm was tested and implemented in C# in the paper, we opted to implement and test a Java application of this sort. This allowed our version to leverage high-resolution timing (*System.nanoTime()*), modern array handling (e.g., *Arrays.copyOf()*), and a modular interface design. These choices enabled us to test the proposed algorithm alongside standard sorting algorithms under identical conditions.

Overall, our approach faithfully reproduces the paper’s logic, while improving modularity, readability, and practical benchmarking.

While following the paper’s structure, we introduced refinements to improve clarity, efficiency, and integration. We built the algorithm around a standard *SortingAlgorithm* interface for seamless benchmarking and we minimized memory usage by using simple counters without unnecessary intermediate structures.

Finally, by embedding the algorithm into an automated benchmark harness, we ensured consistent, scalable evaluation compared to the paper’s original, more manual setup.

IV. Experiments:

To evaluate the performance of each sorting algorithm, we designed a benchmark program that tests algorithms across a range of input sizes and repeats each test multiple times for consistency. Arrays of sizes 10, 100, 1,000, 10,000, and 100,000 were generated with values randomly selected from the range $[-8, 8]$. These arrays were pre-generated and

reused for consistency across algorithms. Each sorting algorithm was run on each array size for 30 independent trials. For every trial, a fresh copy of the input array was passed to the sorting method to avoid side effects from previous runs.

Execution time was measured using *System.nanoTime()*, which provides high-resolution timing. The duration of each sort was recorded in milliseconds and written to a CSV file. This allowed us to compute the average performance of each algorithm over multiple trials, helping to reduce the influence of transient system factors. The benchmarking results provided a reliable comparison of the time complexity and runtime behavior of all sorting algorithms across different input sizes.

V. Results: (Up to 1 Page)

#Inputs	ProposedAlg (<i>lgf</i>)	QuickSort()	BubbleSort (<i>l</i>)	BucketSort (<i>l</i>)	MergeSort (<i>l</i>)	HeapSort()	InsertionSo rt(<i>l</i>)	SelectionS ort(<i>l</i>)
10	0.0017	0.0256	0.002133 ...	0.010066 ...	0.007166 ...	0.0043	0.0012	0.001466 ...
100	0.007133 ...	0.08266...	0.0779	0.033733 ...	0.024066 ...	0.013533 ...	0.0595	0.054233 ...
1000	0.062833 ...	0.367266 ...	0.834	0.1991	0.136566 ...	0.124933 ...	0.474366 ...	0.365266.. ...
10000	0.13866... ...	5.9192	94.88566 ...	0.415566 ...	1.279066 ...	0.82866... ...	8.276233 ...	21.499233 ...
100000	1.058033 ...	172.8136	16077.923 17	2.5786	10.0765	8.138933 ...	766.2407	2093.9490 33

Figure 1: Our results in a simplified table format (measured in ms)

For 10 input sort, ProposedAlgorithm completed the sort in 0.0017ms just shy of 0.0012 and 0.001466 from using insertion and selection sort respectively. For incremental sizes bigger than 10, there starts to become a saving in execution time as the datasets become larger. We see that for 100,000 inputs our ProposedAlgorithm completes the sort in 1.058033 ms which is over 140% more effective than the next best sort mentioned in the paper; bucketSort.

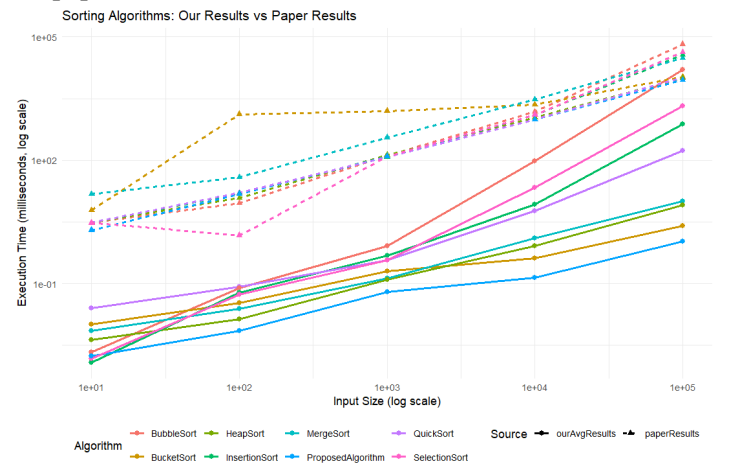


Figure 2: Our results compared with the paper's results

Figure 2 illustrates that our results closely mirror the overall trends observed in the paper. Our results are shown with solid lines, while the paper's results are shown as dashed lines. The elevated execution times in the paper can be attributed to the slower hardware used at the time of its publication. In our evaluation of Figure 1, the ProposedAlgorithm in our testing was superior to the other algorithms, aligning with the results in the paper. For all of the algorithms we observed similar results to what was reported. We observe that results matched our expectation (e.g BubbleSort and InsertionSort exhibit quadratic growth) We can also evaluate that the ProposedAlgorithm has a time complexity of $O(n + k)$ for the environment discussed in the paper.

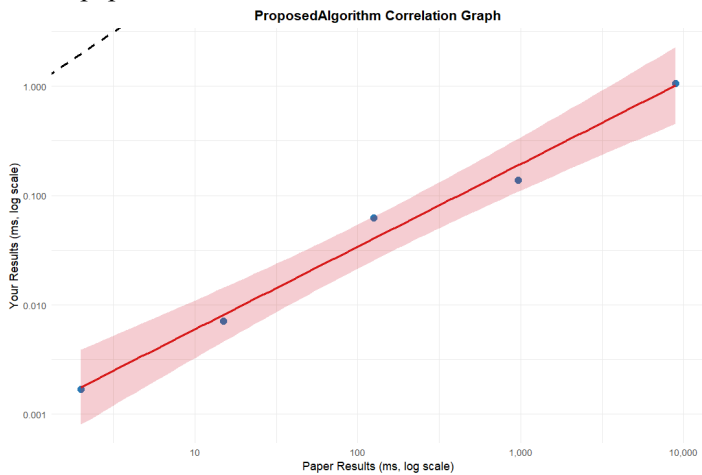


Figure 3: Correlation between our results and the paper's results

Figure 3: the red line indicates a perfect correlation, the shaded red area is our 95% confidence interval for this data set and the blue points represent the actual points for the given array sizes from our results on the y-axis and the paper's results on the x-axis. The dotted line in the top left indicates the perfect 1:1 reference for the algorithm. Despite the limited number of data points, we observe a very strong correlation across all algorithms; even for BucketSort with a 0.994 correlation. The high degree of correlation reinforces our confidence in the validity of our findings for the ProposedAlgorithm, demonstrating its capability to surpass the most established algorithms. We can evaluate from the graph that our results differed slightly compared to the results obtained from the paper but that was to be expected with the advancement in hardware. Importantly, the regression analysis demonstrates that our results scale in a manner consistent with the original study, as evidenced by the strong linear relationship on a log-log scale. The confidence interval around the regression line remains narrow across most of the range, indicating the robustness of the observed trend.

VI. Discussion & Conclusion (Up to ½ Page)

Based on our results, we conclude that the ProposedAlgorithm performs better than the alternative algorithms, on the basis that the inputs are in the range -8 to +8. These results have implications on real world scenarios, not necessarily for small datasets but for larger datasets. The time complexity is evident, algorithms like bubbleSort are unsuitable for large datasets while quickSort, mergeSort and bucketSort are more appropriate. After our testing of the ProposedAlgorithm, we can happily conclude that this algorithm has a tangible impact on scalability and efficiency.

As detailed in the results section, the biggest discrepancies between our findings and the paper's findings were observed in the performance of bucketSort and selectionSort. We attribute the differences in bucketSort to the implementation of the algorithm, however for selectionSort, we are unsure as to what caused the difference. We also couldn't find similar results as the papers' online for both those algorithms. Apart from those two algorithms we were satisfied with our results. In conclusion, the ProposedAlgorithm is an enhanced version of countingSort. It also incorporates the divide and conquer approach of mergeSort which contributes to the efficiency of this algorithm. The ProposedAlgorithm demonstrates better scalability compared to standard mergeSort. It maintains a $O(n + k)$ time complexity, however through our drastic testing, we have never achieved the worst case scenario. The major limitation of this ProposedAlgorithm is the space complexity: $O(n)$ which makes it less suitable for memory constrained environments. The ProposedAlgorithm offers improved performance over all tested algorithms in most scenarios, making it a strong algorithm for sorting large datasets where stability and efficiency are crucial. However, the main limitation of ProposedAlgorithm is its space complexity, which will need consideration in certain scenarios.

VII. Relevant References